

Clase 1 — Mapa de ruta

Paso 1 — Presentación del curso

Objetivo: presentar el propósito del curso, las aplicaciones que se construirán, la modalidad de trabajo, el uso del aula virtual y la responsabilidad de verificar lo producido con inteligencia artificial.

Ítems a tener en cuenta

- Utilizar la [presentación completa](#) para explicar el recorrido y los proyectos.
- Presentar la modalidad práctica y la función del aula virtual.
- Aclarar los roles docentes y la responsabilidad del alumno al revisar y probar.

Paso 2 — Verificación técnica

Objetivo: comprobar que cada alumno pueda acceder al aula virtual, tenga instalados y operativos OpenCode, Node.js y Visual Studio Code, y cree dentro de la carpeta Documentos una carpeta de trabajo que utilizará durante todo el curso.

Ítems a tener en cuenta

- Guiar el ingreso al aula virtual y la localización de los materiales.
- Verificar OpenCode, el modelo gratuito, Node.js, npm y Visual Studio Code.
- Indicar la creación de la carpeta del curso dentro de Documentos y comprobar su apertura.
- Registrar los bloqueos que no puedan resolverse durante el paso.

Paso 3 — Introducción al desarrollo de software asistido por inteligencia artificial generativa

Objetivo: presentar los conceptos principales del desarrollo asistido por inteligencia artificial generativa, incluyendo LLM, modelos, proveedores, prompts, contexto, tokens, agentes, herramientas y responsabilidad humana.

Ítems a tener en cuenta

- Explicar IA generativa, LLM y la diferencia entre modelo, proveedor y aplicación.
- Introducir prompt, contexto y tokens con ejemplos breves.
- Diferenciar un chat de un agente que utiliza herramientas.
- Reforzar los límites de la IA y la responsabilidad humana de verificar.

Paso 4 — Demostración guiada de OpenCode

Objetivo: observar cómo OpenCode trabaja sobre una carpeta local, se conecta con un modelo y realiza acciones mediante herramientas que deben revisarse y verificarse.

Ítems a tener en cuenta

- Presentar OpenCode como aplicación y agente de desarrollo, no como modelo.
- Confirmar la carpeta activa antes de realizar cualquier acción.
- Demostrar la lectura, creación y modificación de un archivo y la ejecución de un comando sencillo.
- Modelar la revisión de permisos, archivos afectados, /undo y /redo.

Paso 5 — Práctica guiada: Proyecto 0

Objetivo: reproducir el flujo observado utilizando OpenCode para transformar el programa del curso en dos archivos Markdown verificables.

Ítems a tener en cuenta

- Guiar la creación de Proyecto 0 y la incorporación del programa del curso.
- Pedir que OpenCode identifique la fuente sin modificarla.
- Generar `resumen-programa.md` y `software-necesario.md` mediante pedidos separados.
- Detenerse después de cada generación para abrir y contrastar el archivo con la fuente.

Paso 6 — Reconstrucción y análisis del Proyecto 0

Objetivo: reconstruir el ciclo de trabajo realizado e identificar qué aportaron el alumno, el documento fuente, el modelo, OpenCode, las herramientas y la verificación.

Ítems a tener en cuenta

- Reconstruir el recorrido desde la carpeta y la fuente hasta los archivos generados.
- Diferenciar la respuesta conversacional de los archivos efectivamente creados.
- Identificar una afirmación que deba verificarse o un aspecto que pueda mejorarse.
- Formular un pedido de corrección y comprobar su efecto en el archivo.

Paso 7 — Presentación del Proyecto 1 y su alcance

Objetivo: comprender el problema, el recorrido obligatorio, los límites y el resultado demostrable del gestor personal de tareas.

Ítems a tener en cuenta

- Mostrar el recorrido esperado mediante una maqueta o demostración breve.
- Presentar las cuatro operaciones y el modelo mínimo de tarea.
- Diferenciar las restricciones estables de las decisiones que podrán evolucionar.
- Explicar que será una experiencia individual, pero sin privacidad ni aislamiento por usuario.
- Anticipar `localStorage` primero y Supabase después; MCP se utilizará durante el desarrollo.

Paso 8 — Presentación de la propuesta inicial de diseño

Objetivo: comprender la propuesta inicial de UX/UI y arquitectura como una hipótesis que orientará el desarrollo y podrá ajustarse mediante decisiones verificadas.

Ítems a tener en cuenta

- Recorrer la pantalla común, sus acciones y sus estados principales.
- Diferenciar interfaz, lógica de tareas y persistencia.
- Comparar el flujo de datos con `localStorage` y con Supabase.
- Mostrar separadamente la conexión de OpenCode mediante MCP.
- Identificar qué elementos son estables y cuáles podrán revisarse durante el proyecto.

Paso 9 — Comparación y definición de la estrategia inicial

Objetivo: comparar alternativas y definir una estrategia inicial común que pueda ajustarse a partir de los resultados obtenidos durante el desarrollo.

Ítems a tener en cuenta

- Comparar brevemente prototipo visual, flujo vertical y avance por funcionalidades.
- Presentar y justificar la estrategia inicial seleccionada por el equipo docente.
- Ordenar los incrementos desde la creación de Next.js hasta la publicación.
- Asociar cada incremento con una comprobación observable.

Paso 10 — Validación colectiva del recorrido

Objetivo: comprobar que los alumnos puedan reconstruir el alcance, las hipótesis de diseño y la estrategia antes de formalizar la especificación inicial.

Ítems a tener en cuenta

- Pedir que los alumnos ordenen colectivamente los incrementos del proyecto.
- Clasificar elementos como estables, provisionales, opcionales o fuera de alcance.
- Comprobar la diferencia entre MCP y el cliente de Supabase.
- Registrar supuestos, dudas y riesgos que deberán validarse durante el desarrollo.

Paso 11 — Formalización de la especificación inicial

Objetivo: establecer una primera versión de la especificación que funcione como contexto común y pueda evolucionar mediante decisiones verificadas durante el desarrollo.

Ítems a tener en cuenta

- Presentar [la plantilla de especificacion-proyecto-1.md](#) como versión 0.1 del proyecto.
- Diferenciar restricciones estables, decisiones provisionales y aspectos fuera de alcance.
- Registrar el diseño, la arquitectura y la estrategia iniciales.
- Definir el primer incremento y su criterio de aceptación.
- Publicar el documento en el aula virtual para utilizarlo en la clase 2.

Paso 12 — Cierre y verificación de salida

Objetivo: comprobar las evidencias producidas, recuperar los aprendizajes centrales y confirmar que cada alumno esté preparado para comenzar el Proyecto 1.

Ítems a tener en cuenta

- Pedir a los alumnos que reconstruyan brevemente el ciclo de trabajo con OpenCode.
- Verificar la carpeta del curso, los archivos del Proyecto 0 y el acceso a [especificacion-proyecto-1.md](#).
- Comprobar que puedan identificar el primer incremento y una función fuera de alcance.
- Registrar cada bloqueo con una próxima acción y un responsable.
- Anticipar que la clase 2 comenzará preparando el contexto y creando Next.js desde cero.

Clase 2 — Mapa de ruta

Paso 1 — Recuperación de la especificación inicial

Objetivo: recuperar la versión 0.1 de la especificación y utilizarla como punto de partida común para comenzar el desarrollo del Proyecto 1.

Ítems a tener en cuenta

- Leer `especificacion-proyecto-1.md` y recuperar sus restricciones estables.
- Identificar las decisiones provisionales y las preguntas pendientes.
- Confirmar `Next.js` desde cero y `localStorage` como única persistencia de la clase.
- Recordar el ciclo de trabajo con `OpenCode` y resolver bloqueos previos.

Paso 2 — Preparación de la carpeta del Proyecto 1

Objetivo: crear y organizar el espacio de trabajo local e incorporar los materiales comunes publicados en el aula virtual.

Ítems a tener en cuenta

- Guiar la descarga de la especificación 0.1 y de la referencia visual desde el aula virtual.
- Indicar la creación de `proyecto-1/documentacion`.
- Guardar `especificacion-proyecto-1.md` dentro de `documentacion`.
- Comprobar que `Visual Studio Code` y `OpenCode` estén abiertos en la carpeta correcta.
- Reforzar que el proyecto se creará desde cero, sin plantilla docente.

La estructura inicial será:

```
proyecto-1/  
└─ documentacion/
```

Paso 3 — Revisión de la especificación vigente

Objetivo: revisar la especificación 0.1 como contexto para `OpenCode` y aclarar lo necesario antes de planificar la primera implementación.

Ítems a tener en cuenta

- Pedir que `OpenCode` lea la especificación sin modificarla y resuma el alcance.
- Comparar el resumen con las restricciones y los criterios de aceptación reales.
- Aclarar solamente las dudas necesarias para iniciar la versión local.
- Registrar preguntas y riesgos sin ampliar el alcance.

Paso 4 — Elaboración del plan de desarrollo con `OpenCode`

Objetivo: transformar el contexto en una secuencia de cambios pequeños que permita alcanzar una versión local completa durante la clase.

Ítems a tener en cuenta

- Pedir que `OpenCode` lea la especificación vigente y proponga un plan antes de modificar archivos.
- Ordenar el plan en incrementos pequeños con una comprobación por incremento.
- Excluir `Supabase`, `MCP` y funciones opcionales.

- Revisar y registrar el plan acordado en la sección de estrategia de la especificación.

El ciclo que se repetirá será:

dar contexto → pedir un plan → revisar → autorizar un cambio pequeño → ejecutar → comprobar

Paso 5 — Creación del proyecto Next.js desde cero

Objetivo: crear una aplicación Next.js nueva dentro de `proyecto-1` y comprobar que funcione antes de comenzar a personalizarla.

Ítems a tener en cuenta

- Modelar el comando de creación y explicar las opciones comunes.
- Revisar el comando y evitar dependencias innecesarias antes de autorizarlo.
- Guiar la instalación, el inicio del servidor y la apertura en el navegador.
- Detener el avance hasta resolver o registrar los errores de creación.

La estructura resultante será:

```
proyecto-1/
├── documentacion/
└── aplicacion/
```

Paso 6 — Exploración del proyecto generado

Objetivo: reconocer la estructura mínima creada por Next.js y localizar los archivos que producen la pantalla visible antes de solicitar cambios.

Ítems a tener en cuenta

- Pedir a OpenCode que explique la estructura sin modificarla.
- Señalar la página principal, los estilos, los recursos y los scripts de `package.json`.
- Comparar la explicación con los archivos reales.
- Identificar qué archivos se modificarán y cómo se verá el cambio.

Paso 7 — Construcción de la interfaz seleccionada

Objetivo: reemplazar la pantalla inicial de Next.js por la única propuesta visual acordada, todavía con datos ficticios y sin persistencia.

Ítems a tener en cuenta

- Guiar la construcción de la única pantalla según la referencia común.
- Incluir formulario, lista ficticia, estados de tarea, eliminación y lista vacía.
- Revisar los archivos modificados y comparar el navegador con la referencia.
- Evitar rediseños o comportamientos todavía no solicitados.

Paso 8 — Definición de las tareas y de la persistencia local

Objetivo: comprender cómo se representarán las tareas y cómo `localStorage` permitirá conservarlas en el navegador.

Ítems a tener en cuenta

- Explicar el modelo de tarea y la colección que mantiene la interfaz.
- Mostrar cómo se guardan y recuperan datos en `localStorage`.

- Acordar una clave común y diferenciar estado temporal de persistencia.
- Pedir un plan específico antes de modificar el código.

Paso 9 — Implementación de creación y listado

Objetivo: permitir que una persona agregue tareas, las vea en pantalla y las recupere después de recargar la aplicación.

Ítems a tener en cuenta

- Guiar la lectura inicial, la creación validada y el renderizado de tareas.
- Comprobar el estado vacío, el título inválido y la limpieza del formulario.
- Recargar la página para verificar la persistencia.
- Mostrar el valor almacenado con las herramientas del navegador.

Paso 10 — Implementación de cambio de estado y eliminación

Objetivo: completar las operaciones obligatorias de la versión local permitiendo cambiar el estado de una tarea y eliminarla.

Ítems a tener en cuenta

- Guiar el cambio de estado y la eliminación por identificador.
- Comprobar que cada operación actualice la interfaz y `localStorage`.
- Probar la eliminación de la última tarea y la persistencia tras recargar.
- Revisar que OpenCode no haya agregado funciones fuera de alcance.

Paso 11 — Verificación y actualización de la especificación

Objetivo: comprobar el recorrido local completo y actualizar la especificación a su versión 0.2 para que represente el producto efectivamente construido.

Ítems a tener en cuenta

- Ejecutar el recorrido completo y revisar consola y terminal.
- Confirmar que `localStorage` sea la única fuente de datos.
- Comparar la implementación con la especificación y distinguir defectos de cambios intencionales.
- Actualizar diseño, arquitectura, decisiones, pendientes e historial de cambios.
- Publicar la versión 0.2 y registrar qué deberá cambiar al incorporar Supabase.

Clase 3 — Mapa de ruta

Paso 1 — Recuperación de la especificación y la versión local

Objetivo: recuperar la especificación 0.2 y comprobar que la versión con `localStorage` funcione antes de cambiar la persistencia.

Ítems a tener en cuenta

- Leer la versión 0.2 e identificar el estado actual y los cambios previstos.
- Ejecutar la versión local y probar las cuatro operaciones después de recargar.
- Comparar la implementación con el diseño y el modelo documentados.
- Resolver los bloqueos antes de iniciar la migración.
- Conservar la versión local como referencia funcional.

Paso 2 — Presentación de la transición hacia Supabase

Objetivo: comprender qué problema resolverá Supabase y qué partes de la aplicación deberán cambiar sin alterar el comportamiento visible ya alcanzado.

Ítems a tener en cuenta

- Comparar `localStorage` con una base de datos en la nube.
- Aclarar que se cambia la persistencia, no la interfaz ni el alcance.
- Introducir operaciones asíncronas y estados de carga y error.
- Exigir datos ficticios y una sola fuente de verdad.

Paso 3 — Introducción a MCP

Objetivo: explicar qué es Model Context Protocol y cómo permite que OpenCode utilice herramientas de Supabase durante el desarrollo.

Ítems a tener en cuenta

- Explicar MCP y los roles de OpenCode, el servidor MCP y Supabase.
- Diferenciar herramientas de lectura de acciones que modifican el proyecto.
- Modelar la revisión de autenticación, alcance, permisos y operaciones propuestas.
- Contrastar la conexión MCP con la conexión de la aplicación en ejecución.

Alumno → OpenCode → servidor MCP → proyecto de Supabase

Aplicación Next.js → cliente de Supabase → proyecto de Supabase

Paso 4 — Conexión de OpenCode con Supabase

Objetivo: configurar y verificar desde cero la conexión de OpenCode con el proyecto de Supabase correspondiente a cada alumno.

Ítems a tener en cuenta

- Guiar la configuración del servidor MCP oficial en el proyecto de práctica correcto.
- Revisar la autenticación, el alcance y los permisos concedidos.
- Comprobar la conexión con una operación de lectura.

- Evitar registrar credenciales o configuraciones sensibles.

Paso 5 — Descripción de la base de datos en lenguaje natural

Objetivo: expresar en lenguaje natural la información y las operaciones que necesita la aplicación para que OpenCode proponga una estructura de base de datos adecuada.

Ítems a tener en cuenta

- Guiar una descripción en lenguaje natural del propósito y del modelo mínimo.
- Precisar campos obligatorios, valores iniciales, identificadores y fechas.
- Recordar que no habrá usuarios ni autenticación.
- Pedir la estructura mínima y una explicación antes de aplicarla.

Paso 6 — Creación y revisión de la tabla mediante MCP

Objetivo: utilizar OpenCode y MCP para crear la estructura acordada en Supabase y verificar que el resultado coincida con la descripción realizada.

Ítems a tener en cuenta

- Revisar tabla, columnas, tipos y valores antes de autorizar la creación.
- Verificar el resultado mediante MCP en modo de lectura.
- Guiar la configuración mínima de RLS y explicar el acceso público sin autenticación.
- Documentar la estructura sin credenciales ni datos sensibles.

Paso 7 — Planificación de la integración de la aplicación

Objetivo: pedir a OpenCode un plan acotado para reemplazar `localStorage` por Supabase manteniendo el comportamiento y el diseño existentes.

Ítems a tener en cuenta

- Pedir que OpenCode lea la especificación y el código e identifique las dependencias de `localStorage`.
- Separar cliente, lectura, mutaciones y estados de carga y error.
- Definir una comprobación por cambio y postergar la eliminación de la versión local.
- Registrar la evolución prevista sin incorporar cambios visuales ni ampliar el alcance.

Paso 8 — Construcción del cliente de Supabase desde cero

Objetivo: instalar y configurar con OpenCode el cliente que utilizará la aplicación Next.js para comunicarse con Supabase en tiempo de ejecución.

Ítems a tener en cuenta

- Guiar la elección e instalación de la dependencia necesaria.
- Diferenciar la clave publicable de las claves secretas y configurar el entorno.
- Pedir a OpenCode que construya el módulo cliente desde cero.
- Comprobar que la aplicación inicie sin exponer valores sensibles.

Paso 9 — Migración de lectura y creación

Objetivo: reemplazar la carga y la creación local de tareas por operaciones contra Supabase.

Ítems a tener en cuenta

- Guiar la lectura remota con estados de carga, vacío y error.
- Migrar la creación manteniendo la validación y la interfaz existentes.

- Probar con datos ficticios y recargar la aplicación.
- Contrastar los registros mediante MCP en modo de lectura.

Paso 10 — Migración de cambio de estado y eliminación

Objetivo: completar la sustitución de `localStorage` implementando en Supabase las operaciones restantes del alcance mínimo.

Ítems a tener en cuenta

- Guiar la actualización de estado y la eliminación en Supabase.
- Reflejar cambios exitosos y comunicar los errores.
- Probar ambas operaciones con varias tareas y después de recargar.
- Retirar `localStorage` y confirmar a Supabase como única fuente.

Paso 11 — Verificación integral y seguridad básica

Objetivo: comprobar el recorrido completo contra Supabase y verificar las condiciones mínimas de seguridad del proyecto sin autenticación.

Ítems a tener en cuenta

- Ejecutar el recorrido completo, incluidos estado vacío, título inválido y recarga.
- Revisar consola, terminal, red y datos mediante MCP.
- Verificar RLS, políticas mínimas y ausencia de claves secretas.
- Recordar que sin autenticación no existe aislamiento por usuario.

Paso 12 — Actualización de la especificación y cierre

Objetivo: actualizar la especificación a su versión 0.3 y dejar una versión candidata preparada para las pruebas y la publicación de la clase 4.

Ítems a tener en cuenta

- Confirmar la aplicación funcional con Supabase como única persistencia.
- Comparar el resultado con la versión 0.2 y justificar los cambios realizados.
- Registrar arquitectura, tabla, políticas, cliente y variables requeridas sin credenciales.
- Documentar verificaciones, problemas pendientes e historial de cambios.
- Publicar la versión 0.3 y anticipar pruebas, publicación y demostración.

Clase 4 — Mapa de ruta

Paso 1 — Recuperación de la especificación y la versión candidata

Objetivo: recuperar la especificación 0.3, comprobar la versión candidata y establecer qué problemas deben resolverse antes de publicarla.

Ítems a tener en cuenta

- Leer la versión 0.3 y recuperar criterios, pendientes y riesgos registrados.
- Comprobar inicio, variables de entorno y conexión con Supabase.
- Ejecutar las cuatro operaciones y confirmar la ausencia de `localStorage`.
- Comparar la aplicación con el estado documentado.
- Identificar y asignar apoyo a los bloqueos prioritarios.

Paso 2 — Congelamiento del alcance

Objetivo: confirmar la versión que será evaluada y evitar que nuevas funcionalidades pongan en riesgo el cierre del proyecto.

Ítems a tener en cuenta

- Confirmar en la especificación el alcance evaluable y los criterios de aceptación.
- Mantener Supabase como única persistencia.
- Aceptar el filtro solo si ya funciona y no compromete el mínimo.
- Impedir ampliaciones y permitir solamente correcciones o cambios justificados.

Paso 3 — Definición de los casos de prueba

Objetivo: convertir los criterios de aceptación en una lista breve de recorridos que otra persona pueda ejecutar y comprobar.

Ítems a tener en cuenta

- Derivar de la especificación casos breves para estado vacío, validación y operaciones obligatorias.
- Incluir recarga, eliminación de la última tarea y un error controlado.
- Especificar el resultado esperado de cada caso.
- Registrar los casos y preparar únicamente datos ficticios.

Paso 4 — Prueba cruzada entre alumnos

Objetivo: permitir que otra persona recorra la aplicación y detecte problemas que su autor podría pasar por alto.

Ítems a tener en cuenta

- Organizar parejas y pedir que ejecuten los casos sin ayuda del autor.
- Evitar cambios durante la primera recorrida.
- Exigir pasos, resultado esperado, resultado observado y evidencia.
- Diferenciar defectos, problemas de uso y preferencias estéticas.

Paso 5 — Registro y priorización de defectos

Objetivo: transformar las observaciones de prueba en una lista de problemas reproducibles ordenada por impacto.

Ítems a tener en cuenta

- Pedir un registro reproducible con título, condiciones, pasos y resultados.
- Incorporar evidencia y frecuencia cuando estén disponibles.
- Clasificar cada defecto como bloqueante, importante o menor.
- Separar mejoras y seleccionar qué corregir durante la clase.

Paso 6 — Diagnóstico asistido por OpenCode

Objetivo: utilizar OpenCode para proponer hipótesis a partir de evidencia concreta antes de autorizar correcciones.

Ítems a tener en cuenta

- Pedir que se entregue a OpenCode la evidencia completa y el código relacionado.
- Solicitar una hipótesis de causa diferenciada de los hechos observados.
- Autorizar un cambio pequeño para un solo defecto.
- Revisar los archivos propuestos y evitar pedidos generales.

Paso 7 — Corrección y prueba de regresión

Objetivo: aplicar las correcciones priorizadas y comprobar que no rompan las operaciones que ya funcionaban.

Ítems a tener en cuenta

- Repetir primero la prueba que fallaba.
- Ejecutar luego la regresión del recorrido completo y revisar los registros.
- Deshacer cambios que introduzcan problemas mayores.
- Registrar la corrección y prohibir mejoras visuales en esta etapa.

Paso 8 — Preparación para la publicación

Objetivo: preparar la aplicación y su configuración para ejecutarse en la plataforma de publicación acordada.

Ítems a tener en cuenta

- Confirmar la plataforma común y ejecutar la compilación local.
- Resolver errores de compilación antes de publicar.
- Guiar las variables de entorno con URL y clave publicable.
- Revisar secretos, archivos excluidos y datos ficticios.

Paso 9 — Publicación de la aplicación

Objetivo: desplegar el Proyecto 1 y obtener una dirección pública que permita recorrerlo fuera de la computadora del alumno.

Ítems a tener en cuenta

- Guiar la vinculación, las variables y el despliegue en la plataforma acordada.
- Pedir la lectura completa de los errores antes de diagnosticarlos con OpenCode.
- Comprobar la dirección pública obtenida.
- Registrar la URL en el aula virtual y verificar que no haya secretos expuestos.

Paso 10 — Verificación de la versión publicada

Objetivo: comprobar que la aplicación publicada funciona de manera independiente del entorno local.

Ítems a tener en cuenta

- Abrir la URL en una sesión nueva y, si es posible, desde otro equipo.
- Probar las operaciones, los estados y la persistencia después de recargar.
- Confirmar la conexión con Supabase y comparar con la versión local.
- Volver a desplegar solo para corregir defectos prioritarios.

Paso 11 — Preparación de la demostración

Objetivo: organizar una presentación breve que muestre el problema, el recorrido principal y una decisión relevante del proceso de desarrollo.

Ítems a tener en cuenta

- Establecer una duración y un formato común para las demostraciones.
- Pedir el recorrido publicado completo y evidencia de persistencia.
- Incluir la transición a Supabase y la diferencia entre MCP y el cliente.
- Solicitar una decisión que haya hecho evolucionar la especificación, un problema resuelto y los pendientes reales.

Paso 12 — Presentaciones, especificación final y cierre

Objetivo: compartir los productos publicados, actualizar la especificación a su versión 1.0 y recuperar los aprendizajes del primer ciclo completo de desarrollo asistido por inteligencia artificial.

Ítems a tener en cuenta

- Coordinar las presentaciones sobre las versiones publicadas.
- Pedir evidencia de prueba y distinguir lo terminado de lo pendiente.
- Actualizar el estado real, la arquitectura, las pruebas, las limitaciones y la URL final.
- Registrar las decisiones y el historial que llevaron desde la versión 0.1 hasta la 1.0.
- Publicar la especificación 1.0 y recuperar aprendizajes para el Proyecto 2.